```
'''
                semantic-RAG
START → POLICY → ROUTER → SEARCH → VALIDATE → END
        |       SQL                 --retry
        |                           --fallback
        block= end

State
→ Node functions
→ Routing functions
→ Add nodes
→ Add edges
→ Add conditional edges
→ Compile
→ Invoke

'''

from typing import TypedDict
from langgraph.graph import StateGraph, START, END
'''
Question: query
Decisions: policy_action, route
Results: context, answer
Controls: validation_status, retry_count
'''
class GraphState(TypedDict,total = False):
    query:str
    policy_action:str
    route:str
    context:str
    answer:str
    validation_state:str
    retry_count:int

#{"query": "What is the borrower's current rating?", "retry_count": 0}
#receives state → performs one job → returns state updates
########## NODE FUNCTIONS ##########
'''
PolicyCheck
'''
def policy_check(state: GraphState) -> dict:
    query = state['query'].lower()
    blocked_terms = ["hack","password","confidential"]
    action = "BLOCK" if any( term in query for term in blocked_terms) else "ALLOW"
    return {"policy_action":action}

def blocked_response(state:GraphState)->dict:
    return { "answer": "This request cannot be processed"}

'''
RouteQuery
'''
def route_query(state:GraphState)->dict:
    query = state['query'].lower()
    sql_terms = ["count","total","average","how many"]
    route = "SQL" if any(term in query for term in sql_terms) else "SEMANTIC"
    return {"route":route}

'''
RAG Search
'''
def rag_search(state:GraphState)->dict:
    query = state['query'].lower()
    context = f"Semantic seach result for: {query}"
```

```
    answer = f"Rag answer generated from :{context}"

    return {
        "context":context,
        "answer":answer,
    }
'''
SQL Search
'''
def sql_search(state:GraphState)->dict:
    query = state['query'].lower()
    context = f"SQL search result for: {query}"
    answer = f"SQL answer generated from :{context}"

    return {
        "context":context,
        "answer":answer,
    }

'''
VALIDATE
'''
def fallback(state: GraphState) -> dict:
    return {
        "answer": "I could not produce a reliable answer from the available information."
    }

########## ADD ROUTING FUNCTIONS ##########
def route_after_policy(state: GraphState)->str:
    if state["policy_action"]=="BLOCK":
        return "blocked_response"
    return "route_query"

def route_after_query(state: GraphState)->str:
    if state["route"]=="SQL":
        return "sql_search"
    return "rag_search"

def route_after_validation(state: GraphState)->str:
    status = state["validation_state"]
    route = state['route'].lower()
    if status == "PASS":
        return END
    if status == "FAIL":
        return "fallback"
    if route=="sql":
        return "sql_search"
    return "rag_search"

########## ADD NODES ##########
workflow = StateGraph(GraphState)
workflow.add("policy_check", policy_check)
workflow.add("blocked_response", blocked_response)
workflow.add("route_query", route_query)
workflow.add("rag_search", rag_search)
workflow.add("sql_search", sql_search)
workflow.add("validate", validate)
workflow.add("fallback", fallback)

########## ADD CONDITIONAL EDGES ##########
'''
Policy validation → BLOCK or ALLOW
Query routing     → RAG or SQL
Answer validation → PASS, RETRY, or FAIL
'''
```

```
workflow.add_edge(START,"policy_check")
workflow.add_conditional_edges("policy_check","route_after_policy")
workflow.add_conditional_edges("route_query","route_after_query")
workflow.add_conditional_edges("route_validation","route_after_validation")

########## ADD NORMAL EDGES ##########
workflow.add_edge("rag_search","validate")
workflow.add_edge("sql_search","validate")
workflow.add_edge("fallback",END)

########## COMPILE and INVOKE ##########
graph = workflow.compile()
result = graph.invoke({
    "query":"whats the borrowers' current rating",
    "retry_count":0
})

```